



# Uniswap: Permissioned Pools Security Review

Cantina Managed review by:

**Robert**, Lead Security Researcher

**Akshay Srivastav**, Security Researcher

August 20, 2026

# Contents

|          |                                                                                                                    |          |
|----------|--------------------------------------------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                                                                                | <b>2</b> |
| 1.1      | About Cantina                                                                                                      | 2        |
| 1.2      | Disclaimer                                                                                                         | 2        |
| 1.3      | Risk assessment                                                                                                    | 2        |
| 1.3.1    | Severity Classification                                                                                            | 2        |
| <b>2</b> | <b>Security Review Summary</b>                                                                                     | <b>3</b> |
| 2.1      | Scope                                                                                                              | 3        |
| <b>3</b> | <b>Findings</b>                                                                                                    | <b>4</b> |
| 3.1      | Medium Risk                                                                                                        | 4        |
| 3.1.1    | Generic <code>BURN_6909</code> steals approved claims                                                              | 4        |
| 3.2      | Low Risk                                                                                                           | 6        |
| 3.2.1    | Permissioned payment actions bypass swap-only controls                                                             | 6        |
| 3.2.2    | <code>PoolManager</code> self-take unwraps raw underlying                                                          | 6        |
| 3.2.3    | <code>PermissionsAdapter</code> should use SafeERC20 for underlying transfers                                      | 7        |
| 3.2.4    | <code>PermissionedPositionManager</code> mints ordinary pools into frozen NFTs                                     | 7        |
| 3.2.5    | <code>PermissionedPositionManager</code> only enforces the hook allowlist when minting                             | 8        |
| 3.2.6    | Permissioned position transfers drop the LP recipient allowlist                                                    | 8        |
| 3.2.7    | Revoked holders can reauthorize liquidity increases                                                                | 9        |
| 3.2.8    | <code>PermissionFlags.hasFlag</code> can incorrectly return <code>true</code> value                                | 9        |
| 3.2.9    | Mapping a missing <code>permissionsAdapterFactory</code> to <code>UnsupportedProtocol</code> bricks ERC20 v4 swaps | 10       |
| 3.2.10   | <code>PermissionsAdapter</code> does not follow the ERC165 standard for checking interface compatibility           | 10       |
| 3.2.11   | Unauthorized <code>UNSUBSCRIBE</code> Removes Subscribers                                                          | 11       |
| 3.2.12   | <code>WithdrawClaim</code> emits unresolved sentinel recipient                                                     | 12       |
| 3.2.13   | <code>PermissionsAdapter.depositForVerification</code> should use SafeERC20 for token transfers                    | 12       |
| 3.3      | Gas Optimization                                                                                                   | 13       |
| 3.3.1    | <code>PermissionedHooks</code> : Redundant <code>Hooks.validateHookPermissions()</code> call                       | 13       |
| 3.3.2    | <code>PermissionedPositionManager._pay()</code> contains duplicate statements in <code>if / else</code> block      | 13       |
| 3.4      | Informational                                                                                                      | 14       |
| 3.4.1    | <code>Permissioned v4 Permit2</code> truncates large amounts                                                       | 14       |
| 3.4.2    | <code>PermissionedHooks</code> trusts wrapper-reported users                                                       | 14       |
| 3.4.3    | Permissioned admins can seize LP NFTs                                                                              | 15       |

# 1 Introduction

## 1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at [cantina.xyz](https://cantina.xyz)

## 1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

## 1.3 Risk assessment

| Severity level     | Impact: High | Impact: Medium | Impact: Low |
|--------------------|--------------|----------------|-------------|
| Likelihood: high   | Critical     | High           | Medium      |
| Likelihood: medium | High         | Medium         | Low         |
| Likelihood: low    | Medium       | Low            | Low         |

### 1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

## 2 Security Review Summary

Uniswap is an open source decentralized exchange that facilitates automated transactions between ERC20 token tokens on various EVM-based chains through the use of liquidity pools and automatic market makers (AMM).

From Apr 13th to Apr 22nd the Cantina team conducted a review of [universal-router](#), [v4-periphery](#) and [v4-hooks-public](#) on commit hashes [0b997cc0](#), [e40c0a3f](#) and [bb21676f](#) respectively. The team identified a total of **20** issues:

Issues Found

| Severity          | Count     | Fixed     | Acknowledged |
|-------------------|-----------|-----------|--------------|
| Critical Risk     | 0         | 0         | 0            |
| High Risk         | 0         | 0         | 0            |
| Medium Risk       | 1         | 1         | 0            |
| Low Risk          | 13        | 13        | 0            |
| Gas Optimizations | 2         | 2         | 0            |
| Informational     | 3         | 2         | 1            |
| <b>Total</b>      | <b>20</b> | <b>18</b> | <b>1</b>     |

### 2.1 Scope

The security review had the following components in scope for [universal-router](#), [v4-periphery](#) and [v4-hooks-public](#) on commit hashes [0b997cc0](#), [e40c0a3f](#) and [bb21676f](#) respectively:

```
├─ universal-router
│   └─ contracts
│       ├── UniversalRouter.sol
│       ├── modules/uniswap/v4/V4SwapRouter.sol
│       └─ types/RouterParameters.sol
├─ v4-hooks-public/src/permissioned-pools/PermissionedHooks.sol
└─ v4-periphery/src/hooks/permissionedPools
    ├── BaseAllowListChecker.sol
    ├── PermissionedPositionManager.sol
    ├── PermissionedV4Router.sol
    ├── PermissionsAdapter.sol
    ├── PermissionsAdapterFactory.sol
    └─ libraries/PermissionFlags.sol
```

## 3 Findings

### 3.1 Medium Risk

#### 3.1.1 Generic `BURN_6909` steals approved claims

**Severity:** Medium Risk

**Context:** (No context files were provided by the reviewer)

**Description:** `PermissionedPositionManager` exposes `Actions.BURN_6909` as a generic action that any external caller can include in a `modifyLiquidities` action bundle. The action decodes an arbitrary `from` address from caller-controlled calldata and forwards it directly to `PoolManager.burn`.

```
if (action == Actions.BURN_6909) {
    (Currency currency, address from, uint256 amount) =
        ↪ params.decodeCurrencyAddressAndUint256();
    poolManager.burn(from, currency.toId(), amount);
    return;
}
```

There is no check that `from` is the current action executor returned by `msgSender()`. There is also no check that the external caller is otherwise authorized by the holder whose claim is burned. Consequently, `PermissionedPositionManager` can be used as a confused deputy after a claim holder authorizes it as a `PoolManager` ERC6909 operator. This authorization is not an unusual or malicious precondition. `withdrawClaim` documents and relies on the same approval model: the caller must hold the claim or must have called `PoolManager.setOperator(permPosm, true)`. The safe external helper constructs `BURN_6909` with `from = msg.sender`, then immediately takes the same amount to the chosen recipient. However, the generic dispatcher remains reachable through `modifyLiquidities`, so an attacker can bypass the constrained helper and provide any claim holder as `from`.

The issue occurs because `PoolManager.burn` authorizes the burn using its immediate caller. In this execution, the immediate caller is `PermissionedPositionManager`, not the attacker who initiated `modifyLiquidities`. `PoolManager.burn` first credits a positive currency delta to `msg.sender`, then calls `_burnFrom(from, id, amount)`. `_burnFrom` permits the burn when `from == msg.sender`, when `msg.sender` is an approved operator for `from` or when sufficient ERC6909 allowance exists. If the victim approved `PermissionedPositionManager` as an operator, the burn succeeds even though the external caller is unrelated to the victim. The credited delta also belongs to `PermissionedPositionManager`, so the attacker can include a following `TAKE` action in the same bundle. This closes the positive delta by transferring the underlying currency to an attacker-controlled recipient. For permissioned adapter currencies, the recipient only needs to pass the underlying token's transfer or compliance checks. For non-permissioned currencies, the transfer is a normal ERC20 or native transfer.

A realistic attack is straightforward. Alice is the admin of a permissioned token adapter. She force-exits a non-compliant LP with `unwindPosition`. The LP cannot receive the underlying permissioned token and Alice is also currently unable to receive it directly because of the token's transfer restrictions. `unwindPosition` therefore preserves the value by minting Alice an ERC6909 claim in `PoolManager`. That claim is not a bookkeeping marker with no value. It is the withdrawable representation of the underlying tokens owed by the pool manager. Alice later wants to redeem that claim through `withdrawClaim`, so she follows the documented requirement and calls `PoolManager.setOperator(address(permissionedPositionManager), true)`. This grants `PermissionedPositionManager` operator rights over Alice's ERC6909 balances in the pool manager. Alice has not approved Bob, the attacker. She has only approved the position manager that the protocol tells her to use. Bob watches for this approval and for Alice's ERC6909 balance. He then submits one `modifyLiquidities` call to `PermissionedPositionManager` with two actions. The first action is `BURN_6909(adapterCurrency, Alice, claimAmount)`. The second action is `TAKE(adapterCurrency, Bob, claimAmount)`. `PoolManager` sees the burn as a call from `PermissionedPositionManager`, which Alice approved as an operator, so the burn succeeds. The burn creates a positive delta for

`PermissionedPositionManager` . The following `TAKE` action closes that delta by transferring the underlying currency to Bob. After the transaction, Alice's ERC6909 claim balance is zero. Alice can no longer call `withdrawClaim` because the claim was already burned. Bob receives the same amount of underlying tokens that Alice should have received. The attack does not require Bob to own Alice's NFT, control any permissioned adapter, be approved by Alice directly or compromise Alice's key.

This is not limited by position ownership or by the permissioned pool's LP authorization checks. The vulnerable action is handled before the base position-manager dispatcher and does not reference an NFT, pool position, hook allowlist or liquidity permission. The only meaningful constraints are that the victim has a burnable ERC6909 claim, the victim approved `PermissionedPositionManager` as an ERC6909 operator or allowance spender and the attacker's chosen recipient can receive the underlying currency.

The issue was validated with a focused Foundry proof of concept that can be found at <https://gist.github.com/r0bert-ethack/4424d839e516daf1b66b26bd964cbe22>. The test creates a real permissioned-currency claim through `unwindPosition` , has the victim call `PoolManager.setOperator(address(lpm), true)` , then has an unrelated attacker call `modifyLiquidities` with generic `BURN_6909` and `TAKE` actions. The test passed with:

```
forge test --match-path test/zz_POCS/Burn6909ConfusedDeputy.t.sol --match-test
↳ test_poc_attackerStealsVictimClaimThroughGenericBurn6909 -vvvv
```

The trace shows `PermissionedPositionManager` calling `PoolManager.burn(victim, id, claim)` , with the ERC6909 burn event emitted by the pool manager showing `caller: PermissionedPositionManager` . The next action calls `PoolManager.take(currency, attacker, claim)` . The final assertions show that the victim's ERC6909 balance is zero and the attacker received exactly the burned claim amount as underlying currency.

The impact is direct theft of ERC6909 claim balances from users who approved `PermissionedPositionManager` for claim withdrawal. With `setOperator` , the approval is account-wide for the pool manager's ERC6909 claims, so the exposed value is not limited to one claim id. A single standing approval can expose all current and future ERC6909 claims held by that account that can be burned through the vulnerable dispatcher and then taken to an attacker-controlled recipient. If a user approved only a per-id ERC6909 allowance instead of operator status, the exposed value is limited to that allowance and token id, but the same confused-deputy burn still applies. The maximum loss for an affected account is the full claim balance covered by the victim's approval and by currencies whose underlying transfer can be delivered to the attacker's recipient. In practical terms, this can be the entire value returned from forced exits or other operations that intentionally fall back to ERC6909 claims when direct token delivery is unavailable.

**Recommendation:** Do not expose `BURN_6909` as an unconstrained public action in `PermissionedPositionManager._handleAction` . The smallest safe fix is to require the burned claim holder to be the current action executor:

```
if (action == Actions.BURN_6909) {
    (Currency currency, address from, uint256 amount) =
    ↳ params.decodeCurrencyAddressAndUint256();
    if (from != msgSender()) revert Unauthorized();
    poolManager.burn(from, currency.toId(), amount);
    return;
}
```

A stronger design is to remove generic `BURN_6909` support from the public action dispatcher and keep claim burning behind `withdrawClaim` . If internal reuse is needed, set a transient authorized claim-withdrawer value inside `withdrawClaim` before unlocking the pool manager. Then permit `BURN_6909` only while that transient authorization is active and only for the expected `(currency, from, amount)` tuple.

If `UniversalRouter` can be configured with this permissioned position manager, harden `_checkV4PositionManagerCall` as defense in depth. Its current guard rejects several position-altering actions but does not reject `BURN_6909` . Calls routed through the universal router should reject

**BURN\_6909** when the configured v4 position manager is permissioned and ideally should allow only the minimum action set needed for the intended migration or minting operation.

**Uniswap:** Fixed in PR 555.

**Cantina Managed:** Fix verified.

## 3.2 Low Risk

### 3.2.1 Permissioned payment actions bypass swap-only controls

**Severity:** Low Risk

**Context:** `PermissionedV4Router.sol#L24-L43`

**Description:** `PermissionedV4Router._pay()` treats `SWAP_ALLOWED` as enough to handle a permissioned token. It does not check that the current action is part of a real swap and it does not check `swappingEnabled()`.

That matters because `V4Router` still exposes standalone `SETTLE`, `SETTLE_ALL`, `TAKE` and `TAKE_ALL` actions. Those actions can reach `_settle()`, then `_pay()`, without ever executing `PermissionedHooks.beforeSwap()`. The hook-side policy checks only run during an actual swap callback. Consequently, a caller with `SWAP_ALLOWED` can still use the permissioned v4 router as a generic wrap or unwrap surface through `V4_SWAP` plans that never perform a swap.

This weakens the intended boundary of the permissioned swap integration. Disabling swaps in the hook no longer fully disables permissioned token handling through the router and the router accepts behavior that is broader than "swap only".

**Recommendation:** Require explicit swap context before permissioned payment actions are allowed to proceed.

Consider tracking whether execution is currently inside a hook-validated swap and reject permissioned `_pay()` calls otherwise. As additional defense, mirror the `swappingEnabled()` check in the router so permissioned balance handling cannot continue after swaps have been disabled.

**Uniswap:** Fixed in PR 552.

**Cantina Managed:** Partially fixed. The code fixes the pause-bypass portion of the finding, but it does not fully enforce the original "swap only" boundary. `PermissionedV4Router._pay()` now checks `permissionsAdapter.swappingEnabled()` before accepting permissioned payment handling, so standalone payment actions can no longer wrap permissioned tokens through the router while swaps are disabled. This directly addresses the initial concern that disabling swaps in the hook did not disable permissioned token handling in the router. However, `_pay()` still does not require that the current action sequence is inside, or caused by a hook-validated swap. A `V4_SWAP` command can still contain only `SETTLE` and `TAKE` actions. With `swappingEnabled() == true` and `SWAP_ALLOWED`, that sequence reaches `_settle()`, `_pay()`, `wrapToPoolManager()`, `settle()` and then `take()` without any `SWAP_EXACT_*` action and without executing `PermissionedHooks.beforeSwap()`. This no-swap `SETTLE` plus `TAKE` plan succeeds when swapping is enabled and reverts when swapping is disabled. If integrators, issuers or compliance logic assume the router is only a swap surface, that assumption is false.

### 3.2.2 `PoolManager` self-take unwraps raw underlying

**Severity:** Low Risk

**Context:** `PermissionsAdapter.sol#L99-L114`

**Description:** `PermissionsAdapter._update()` unwraps whenever `from == POOL_MANAGER`. It does not exclude the self-transfer case where `to == POOL_MANAGER` as well. In that case the transfer is not a no-op. `_unwrap(POOL_MANAGER, amount)` burns the adapter balance and transfers the raw underlying token into `PoolManager`.

This changes the asset type inside the settlement contract. The permissioned v4 stack treats the verified adapter address as the protected currency, but a self-take removes that adapter representation and leaves raw underlying in `PoolManager` instead. `V4Router` still allows arbitrary `TAKE` recipients, so a route can point adapter output at `address(poolManager)` and reach this conversion through normal action execution.

This does not directly pay an attacker, but it breaks the adapter-based accounting boundary that the permissioned design relies on. Consequently, later logic can mis-handle or strand balances if it assumes `PoolManager` only holds the adapter currency for that pool.

The issue does not require `PoolManager` to have adapter-level permission such as `SWAP_ALLOWED` or `LIQUIDITY_ALLOWED`. `PermissionsAdapter._update()` only checks that the sender is `POOL_MANAGER` before `_unwrap()`, so the recipient's adapter permission is irrelevant when `from == to == POOL_MANAGER`. The raw underlying transfer still has to succeed. For a normal ERC20 this requires no setup, while a recipient-gated permissioned token must separately allow `PoolManager` as a raw-token recipient.

**Recommendation:** Do not unwrap on a pool-manager self-transfer. Revert, or skip `_unwrap()`, when `from == to == POOL_MANAGER`. As additional defense, reject `address(poolManager)` as a `TAKE` recipient for verified adapter currencies in the official wrappers so this conversion cannot be reached through normal user-facing plans.

**Uniswap:** Fixed in [PR 553](#).

**Cantina Managed:** Fix verified.

### 3.2.3 `PermissionsAdapter` should use `SafeERC20` for underlying transfers

**Severity:** Low Risk

**Context:** [PermissionsAdapter.sol#L113](#)

**Description:** `PermissionsAdapter._unwrap` burns the adapter balance and then calls `PERMISSIONED_TOKEN.transfer(account, amount)` through the standard `IERC20` interface. That assumes every permissioned token follows the exact ERC20 return-value shape expected by Solidity.

Some widely used tokens, including USDT-style tokens, do not return a boolean from `transfer`. Because `PERMISSIONED_TOKEN` is typed as an `IERC20`, Solidity expects return data that can be decoded as `bool`. A no-return token can therefore make `_unwrap` revert even though the token transfer itself would otherwise have succeeded.

The opposite compatibility issue is false-returning tokens. If a token returns `false` instead of reverting, a raw transfer call is easy to mishandle unless the return value is checked consistently. The adapter should use the standard safe-transfer wrapper for `IERC20` transfers so no-return tokens are accepted and false-returning tokens revert cleanly.

**Recommendation:** Use `SafeERC20.safeTransfer` for the underlying `IERC20` transfer in `_unwrap` instead of calling `PERMISSIONED_TOKEN.transfer(...)` directly. The burn and transfer can stay in the same function; if `safeTransfer` reverts, the burn is rolled back with the rest of the transaction.

**Uniswap:** Fixed in [PR 534](#).

**Cantina Managed:** Fix verified.

### 3.2.4 `PermissionedPositionManager` mints ordinary pools into frozen NFTs

**Severity:** Low Risk

**Context:** [PermissionedPositionManager.sol#L59-L68](#)



**Description:** `PermissionedPositionManager.transferFrom` only allows the owner of a verified permissions adapter in the pool to move a position NFT. When neither pool currency is a verified adapter, `_getOwner` returns `address(0)` for both sides. At the same time, `_mint` still accepts those same ordinary pools because `_checkAllowedHook` and `_checkRecipientAllowed` both return success when `verifiedPermissionsAdapterOf(...)` is zero.

Consequently, this manager can mint positions for ordinary pools that have no adapter-backed admin at all, but later neither the owner nor an approved operator can transfer the NFT. `safeTransferFrom` is disabled outright, so the receipt token is frozen once minted.

**Recommendation:** Reject mints unless at least one pool currency is a verified permissions adapter. If ordinary pools are meant to stay supported, keep the admin-only transfer rule only for positions that actually contain a verified adapter and preserve normal ERC721 transfer behavior otherwise.

**Uniswap:** Fixed in [PR 550](#).

**Cantina Managed:** Fix verified.

### 3.2.5 `PermissionedPositionManager` only enforces the hook allowlist when minting

**Severity:** Low Risk

**Context:** [PermissionedPositionManager.sol#L78-L95](#)

**Description:** `PermissionedPositionManager` exposes `setAllowedHook(...)` as a live policy control for verified adapters, but it only enforces that registry inside `_mint`. Once a permissioned position already exists, later liquidity increases go through the inherited `PositionManager._increase` and `_increaseFromDeltas` paths. Those paths call `poolManager.modifyLiquidity(...)` directly and never re-check whether the stored pool hook is still allowed for the permissioned adapters in the position.

Consequently, `setAllowedHook(currency, hook, false)` is not an effective emergency control for existing positions. It only blocks fresh mints. Position owners and approved operators can still add more liquidity to already-minted NFTs on the revoked hook through the official permissioned manager. That weakens the integration boundary around `IPermissionsAdapter`, because the surrounding stack treats the adapter owner as the live policy authority for wrappers, allowlists, swapping status, and hook selection.

**Recommendation:** Re-check the current pool hook before every permissioned liquidity increase, not only during minting. Override `_increase(...)` and `_increaseFromDeltas(...)` in `PermissionedPositionManager`, load the stored `PoolKey` for the position and revert unless `_checkAllowedHooks(poolKey)` still succeeds.

**Uniswap:** Fixed in [PR 537](#).

**Cantina Managed:** [PR 537](#) fixes this by adding hook validation in both `_increase()` and `_increaseFromDeltas()`.

### 3.2.6 Permissioned position transfers drop the LP recipient allowlist

**Severity:** Low Risk

**Context:** [PermissionedPositionManager.sol#L59-L68](#)

**Description:** `PermissionedPositionManager` enforces the LP recipient allowlist when minting, but `transferFrom(...)` does not reuse that check. It only verifies that `msg.sender` is the owner of one of the verified permission adapters in the position and then forwards the transfer to `super.transferFrom(...)`.

This breaks the recipient-side policy that the rest of the permissioned stack appears to rely on. A position that could not be minted to `to` can still be moved there later by an adapter admin. Once that happens, the unauthorized holder controls the NFT and can manage the existing position through the normal owner-approved `PositionManager` surfaces. In a pool where both sides are permissioned, either adapter owner can do this unilaterally even though the position represents both assets.

**Recommendation:** Apply the same recipient check in `transferFrom(...)` that `_mint(...)` already applies during creation. Before delegating to `super.transferFrom(...)`, call `_checkRecipientAllowed(...)` for each verified adapter in the position and revert unless the recipient has `LIQUIDITY_ALLOWED`.

If adapter admins are intended to retain seizure powers, the seized position should still be transferred only to an address that satisfies the same liquidity recipient permission checks. Otherwise the seizure mechanism becomes a way to bypass the recipient allowlist.

**Uniswap:** Fixed in [PR 542](#) and [PR 543](#).

**Cantina Managed:** [PR 542](#) and [PR 543](#) fix this by removing transferability of permissioned positions and replacing it with an `unwindPosition()` mechanism.

### 3.2.7 Revoked holders can reauthorize liquidity increases

**Severity:** Low Risk

**Context:** [PermissionedPositionManager.sol#L78-L102](#)

**Description:** `PermissionedPositionManager` only enforces the position holder's `LIQUIDITY_ALLOWED` status during `_mint`, where it calls `_checkRecipientAllowed` on the new owner. It does not apply the same owner-side check when liquidity is added to an existing position. Later increases reuse the inherited `PositionManager._increase(...)` and `_increaseFromDeltas(...)` paths, which only require `onlyIfApproved(msgSender(), tokenId)`.

That means the permission check shifts from the NFT holder to the caller. `PermissionedHooks.beforeAddLiquidity` and `PermissionedPositionManager._pay` both validate `msgSender()`, not the current `ownerOf(tokenId)`. Since ERC721 approval remains available after revocation, a holder who has already lost `LIQUIDITY_ALLOWED` can still call `approve(...)` or `setApprovalForAll(...)` and authorize any allowlisted operator to increase the position on their behalf.

Consequently, revoking `LIQUIDITY_ALLOWED` from an address does not actually stop that address from accumulating more permissioned liquidity in positions it already owns. It only stops the address from calling the increase directly.

**Recommendation:** Re-check the current position owner before every permissioned liquidity increase, not only at mint time. Override the increase paths in `PermissionedPositionManager`, load the pool key and current `ownerOf(tokenId)` and call `_checkRecipientAllowed` for each verified adapter before delegating to the inherited logic.

If delegated management for revoked holders is intended, specify that exception explicitly and document that `LIQUIDITY_ALLOWED` is an operator-only gate after mint.

**Uniswap:** Fixed in [PR 554](#).

**Cantina Managed:** Fix verified.

### 3.2.8 `PermissionFlags.hasFlag` can incorrectly return `true` value

**Severity:** Low Risk

**Context:** [PermissionFlags.sol#L28-L30](#)

**Description:** The `hasFlag` function of `PermissionFlags` library can incorrectly return `true` when two bits are set for either `permissions` or `flag` values.

Example, a contract requires both `SWAP_ALLOWED` and `LIQUIDITY_ALLOWED` permission for accounts but an account only has `SWAP_ALLOWED` permission, in this case the `PermissionFlags.hasFlag` function will incorrectly return `true` as return value.

### Proof of Concept:

```
contract PermissionFlagsLibTest is Test {
    using PermissionFlags for PermissionFlag;

    function test_audit_permissionFlags_hasFlag() public pure {
        PermissionFlag permissions = PermissionFlag.wrap(0x0001); // SWAP_ALLOWED only
        PermissionFlag flag = PermissionFlag.wrap(0x0003); // SWAP_ALLOWED and
        ↪ LIQUIDITY_ALLOWED

        assertTrue(permissions.hasFlag(flag));
    }
}
```

### Recommendation:

```
function hasFlag(PermissionFlag permissions, PermissionFlag flag) internal pure returns
    ↪ (bool) {
-     return PermissionFlag.unwrap(and(permissions, flag)) != 0;
+     return PermissionFlag.unwrap(and(permissions, flag)) !=
    ↪ PermissionFlag.unwrap(flag);
}
```

**Uniswap:** Fixed in [PR 535](#).

**Cantina Managed:** [PR 535](#) fixes this by removing the unused `hasFlag` library function entirely.

### 3.2.9 Mapping a missing `permissionsAdapterFactory` to `UnsupportedProtocol` bricks ERC20 v4 swaps

**Severity:** Low Risk

**Context:** [DeployUniversalRouter.s.sol#L36-L48](#)

**Description:** `PermissionedV4Router` treats `PERMISSIONS_ADAPTER_FACTORY == address(0)` as the disabled mode for permissioned pools. In that mode it skips the adapter lookup and falls back to `_payStandard`. `DeployUniversalRouter.run` breaks that assumption by rewriting every zero-valued protocol address to `UnsupportedProtocol`.

All current deployment parameter files leave `permissionsAdapterFactory` unset. For example, `script/deployParameters/DeployMainnet.s.sol:16-18` passes `address(0)`. After `mapUnsupported` runs, the deployed `V4SwapRouter` no longer sees a zero factory. During any ERC20 `V4_SWAP` settle, `PermissionedV4Router._pay` and `_mapSettleAmount` call `verifiedPermissionsAdapterOf(...)` on the dummy contract. `UnsupportedProtocol` reverts in its fallback, so the router cannot settle normal ERC20 input into the v4 `PoolManager`.

This turns “permissioned pools are not configured” into “non-permissioned ERC20 v4 swaps revert in production.” The issue is easy to miss because the local integration helper deploys with the real zero address, so tests exercise the working branch and not the deployed one.

**Recommendation:** Keep `permissionsAdapterFactory` as the real zero address until a live factory exists. The smallest safe fix is to exclude this field from `mapUnsupported`.

**Uniswap:** Fixed in [PR 483](#).

**Cantina Managed:** Fix verified.

### 3.2.10 `PermissionsAdapter` does not follow the ERC165 standard for checking interface compatibility

**Severity:** Low Risk

**Context:** `PermissionsAdapter.sol#L66-L72`

**Description:** The `PermissionsAdapter._updateAllowListChecker` function simply calls the `newAllowListChecker.supportsInterface(...)` function to check interface compatibility. However this is not the correct way of interface validation.

The `ERC165` states that the caller should first call `contract.supportsInterface(0x01ffc9a7)` and `contract.supportsInterface(0xffffffff)` and validate the results before actually calling the `contract.supportsInterface(interfaceId)`.

**Recommendation:** Consider using the Openzeppelin's `ERC165Checker` library for interface compatibility validation.

**Uniswap:** Fixed in [PR 549](#).

**Cantina Managed:** Fix verified.

### 3.2.11 Unauthorized `UNSUBSCRIBE` Removes Subscribers

**Severity:** Low Risk

**Context:** *(No context files were provided by the reviewer)*

**Description:** `PermissionedPositionManager` adds a custom `Actions.UNSUBSCRIBE` case to its action dispatcher. Any external caller can reach this dispatcher by calling `modifyLiquidities`. The custom action decodes an arbitrary `tokenId` from caller-controlled calldata and calls the internal `_unsubscribe` function when the position currently has a subscriber.

```
if (action == Actions.UNSUBSCRIBE) {
    uint256 tokenId = abi.decode(params, (uint256));
    if (positionInfo[tokenId].hasSubscriber()) _unsubscribe(tokenId);
    return;
}
```

This bypasses the authorization check that the normal external unsubscription function enforces. `Notifier.unsubscribe` is restricted by `onlyIfApproved(msg.sender, tokenId)`, so only the NFT owner, an approved address or an approved operator can remove a position subscriber through the public notifier interface. The internal `_unsubscribe` function intentionally does not repeat that check. It only verifies that a subscriber exists, clears the subscriber flag, deletes the `subscriber[tokenId]` entry, optionally calls `notifyUnsubscribe` and emits `Unsubscription`. The custom action therefore creates an unauthenticated unsubscription primitive. `modifyLiquidities` stores the external caller in the reentrancy locker and `msgSender()` would return that caller during the unlock callback. However, the `UNSUBSCRIBE` action never checks `msgSender()` against the position owner or approval state. As a result, any account can remove the subscriber from any permissioned position by submitting an action bundle containing only `Actions.UNSUBSCRIBE` and the target `tokenId`. This matters because subscribers are not only passive UI conveniences. The position manager's notifier system exists so external contracts can receive `notifySubscribe`, `notifyUnsubscribe`, `notifyModifyLiquidity` and `notifyBurn` callbacks for a position. Integrators can use those callbacks for monitoring, automation, accounting, compliance checks, strategy management, risk alerts or off-chain reconciliation. An unauthorized unsubscription silently detaches that integration from the position. Future liquidity changes and burns will no longer notify the expected subscriber.

A realistic example is an LP who attaches a subscriber contract that monitors a permissioned position and triggers operational alerts when liquidity changes. An attacker does not need to own the LP's NFT and does not need any ERC721 approval. The attacker calls `PermissionedPositionManager.modifyLiquidities` with a single action: `UNSUBSCRIBE(tokenId)`. The pool manager unlock callback executes the permissioned manager's custom dispatcher, which calls `_unsubscribe(tokenId)` directly. The subscriber is deleted while the NFT remains owned by the LP. The LP's later liquidity changes no longer reach the monitoring contract. The same issue can affect automation. If a subscriber is responsible for updating external accounting, rebalancing metadata, compliance bookkeeping or keeper state, an attacker can remove the

subscriber before the position changes again. The direct on-chain asset custody is not transferred by this action, so the severity depends on how heavily downstream systems rely on subscriber callbacks. When subscribers are operationally important, this is a meaningful denial of service and monitoring bypass against another user's position.

**Recommendation:** Require the current action executor to be authorized for the position before calling `_unsubscribe` from the custom dispatcher. The check should mirror the authorization used by the public notifier function:

```
if (action == Actions.UNSUBSCRIBE) {
    uint256 tokenId = abi.decode(params, (uint256));
    if (!_isApprovedOrOwner(msgSender(), tokenId)) revert NotApproved(msgSender());
    if (positionInfo[tokenId].hasSubscriber()) _unsubscribe(tokenId);
    return;
}
```

This preserves the intended use inside `unwindPosition`. Before entering the pool-manager unlock callback, `unwindPosition` sets `getApproved[tokenId] = msg.sender` for the permissioned adapter admin. That pre-approval already exists so the following `BURN_POSITION` action can pass `onlyIfApproved(msgSender(), tokenId)`. The same pre-approval will also allow the fixed `UNSUBSCRIBE` action to succeed during `unwindPosition`.

**Uniswap:** Fixed in [PR 556](#).

**Cantina Managed:** Fix verified.

### 3.2.12 `WithdrawClaim` emits unresolved sentinel recipient

**Severity:** Low Risk

**Context:** (No context files were provided by the reviewer)

**Description:** `withdrawClaim` accepts the same recipient sentinel values used by `Actions.TAKE`: `address(1)` resolves to the current action executor and `address(2)` resolves to the position manager contract. The function builds a `BURN_6909` plus `TAKE` action bundle with the caller-provided `to`, then emits `ClaimWithdrawn` with that original value.

The actual token delivery is handled later by the inherited `TAKE` action, which resolves sentinel recipients before calling `poolManager.take`. Consequently, the event can report `address(1)` or `address(2)` even though the underlying tokens were delivered to a different concrete address. Off-chain accounting, reconciliation, or monitoring that relies on `ClaimWithdrawn.to` can attribute the withdrawal to the sentinel address instead of the real recipient.

This does not change on-chain balances and does not create a direct token-loss primitive. The impact is incorrect event data for claim withdrawals that use recipient sentinels.

**Recommendation:** Resolve the recipient before emitting `ClaimWithdrawn`, and use the same resolved address that the `TAKE` action will use. For example, compute `address resolvedTo = _mapRecipient(to)` before constructing the action parameters, pass `resolvedTo` into the `TAKE` action and emit `ClaimWithdrawn(currency, msg.sender, resolvedTo, amount)`.

**Uniswap:** Fixed in [PR 557](#).

**Cantina Managed:** Fix verified.

### 3.2.13 `PermissionsAdapter.depositForVerification` should use SafeERC20 for token transfers

**Severity:** Low Risk

**Context:** (No context files were provided by the reviewer)

**Description:** `PermissionsAdapter.depositForVerification` calls `PERMISSIONED_TOKEN.transferFrom(...)` through the standard `IERC20` interface. It assumes every permissioned token follows the exact ERC20 return-value shape expected by Solidity.

Some widely used tokens, including USDT-style tokens, do not return a boolean from `transferFrom`. It is also possible for an ERC20 token to return `false` instead of reverting. Both these scenarios lead to mishandling of `transferFrom` call and tokens.

**Recommendation:** Use `SafeERC20.safeTransferFrom` for the underlying IERC20 transfer.

**Uniswap:** Fixed in [PR 558](#).

**Cantina Managed:** Fix verified.

### 3.3 Gas Optimization

#### 3.3.1 `PermissionedHooks` : Redundant `Hooks.validateHookPermissions()` call

**Severity:** Gas Optimization

**Context:** `BaseHook.sol#L18-L20`, `PermissionedHooks.sol#L30-L33`

**Description:** The `constructor` of `PermissionedHooks` contract performs a `Hooks.validateHookPermissions()` call. As `PermissionedHooks` contract inherits `BaseHook` contract the `validateHookPermissions()` call is already performed by `BaseHook.constructor`.

**Recommendation:** Consider removing the redundant operation.

```
constructor(IPoolManager manager, IPermissionsAdapterFactory permissionsAdapterFactory)
↳ BaseHook(manager) {
    PERMISSIONS_ADAPTER_FACTORY = permissionsAdapterFactory;
-   Hooks.validateHookPermissions(this, getHookPermissions());
}
```

**Uniswap:** Fixed in [PR 24](#).

**Cantina Managed:** Fix verified.

#### 3.3.2 `PermissionedPositionManager._pay()` contains duplicate statements in `if / else` block

**Severity:** Gas Optimization

**Context:** `PermissionedPositionManager.sol#L116-L141`

**Description:** The `PermissionedPositionManager._pay()` function contains same and duplicate statements in both `if` and `else` block. Same operations get performed whether the code execution takes the `if` or the `else` path. These duplicate statements can be put outside the `if/else` block.

**Recommendation:**

```
function _pay(Currency currency, address payer, uint256 amount) internal virtual
↳ override {
//...
+   // Check liquidity permission for the actual user
+   if (!permissionsAdapter.isAllowed(msgSender(), PermissionFlags.LIQUIDITY_ALLOWED))
↳ {
+       revert Unauthorized();
+   }
    if (payer == address(this)) {
-       // Check liquidity permission for the actual user
-       if (!permissionsAdapter.isAllowed(msgSender(),
↳ PermissionFlags.LIQUIDITY_ALLOWED)) {
```



```

-         revert Unauthorized();
-     }
    Currency.wrap(permissionedToken).transfer(address(permissionsAdapter), amount);
-     permissionsAdapter.wrapToPoolManager(amount);
    } else {
-         // Check liquidity permission for the actual user
-         if (!permissionsAdapter.isAllowed(msgSender(),
↪ PermissionFlags.LIQUIDITY_ALLOWED)) {
-             revert Unauthorized();
-         }
        // token is a permissioned token, wrap the token
        permit2.transferFrom(payer, address(permissionsAdapter), uint160(amount),
↪ permissionedToken);
-         permissionsAdapter.wrapToPoolManager(amount);
    }
+     permissionsAdapter.wrapToPoolManager(amount);
}

```

**Uniswap:** Fixed in [PR 548](#).

**Cantina Managed:** Fix verified.

## 3.4 Informational

### 3.4.1 `Permissioned v4 Permit2` truncates large amounts

**Severity:** Informational

**Context:** [V4SwapRouter.sol#L26-L34](#)

**Description:** `V4SwapRouter._payPermissionedFromPayer()` passes `uint160(amount)` into `PERMIT2.transferFrom()`, then immediately calls `permissionsAdapter.wrapToPoolManager(amount)` with the original `uint256` amount. Unlike `Permit2Payments.payOrPermit2Transfer()`, this helper does not check that `amount` fits into `uint160` before narrowing it.

When `amount > type(uint160).max`, `Permit2` only pulls the truncated low 160 bits from the payer. `wrapToPoolManager` still tries to mint the full `amount`. If the adapter already holds enough spare underlying balance, that existing inventory can cover the missing high bits and the settle succeeds after undercharging the payer. If the adapter does not hold enough spare balance, the call reverts. In both cases, the permissioned v4 integration does not fail closed at the cast boundary the way the repo's standard `Permit2` helper does.

This same raw cast also appears in `lib/v4-periphery/src/hooks/permissionedPools/PermissionedPositionManager.sol`, so permissioned liquidity settles inherit the same truncation behavior.

**Recommendation:** Apply the same checked cast used by `Permit2Payments` before every permissioned `Permit2` transfer that accepts a `uint256` amount. Replace `uint160(amount)` with `amount.toUint160()` or an equivalent explicit overflow check, then mirror that change in `PermissionedPositionManager`.

**Uniswap:** Acknowledged as a confirmed trade-off carried over from the inherited `PositionManager._pay`, which has the [inline justification](#):

"Casting from `uint256` to `uint160` is safe due to limits on the total supply of a pool".

**Cantina Managed:** Acknowledged.

### 3.4.2 `PermissionedHooks` trusts wrapper-reported users

**Severity:** Informational

**Context:** [PermissionedHooks.sol#L75-L98](#)

**Description:** `PermissionedHooks` does not authenticate the user whose permissions it checks. `_verifyAllowlist` asks the wrapper for `sender.msgSender()`, then `_isAllowed` separately checks only that the wrapper address is present in `allowedWrappers`.

This makes wrapper approval equivalent to trusting that wrapper to report the user honestly. A whitelisted wrapper can return an allowlisted address even when the real caller is not approved. It can also return different addresses across the two `sender.msgSender()` calls, so the `currency0` and `currency1` checks do not even have to evaluate the same subject.

The in-repo router and position manager are currently safe because their `msgSender()` comes from `ReentrancyLock` state instead of arbitrary wrapper logic. The exposed integration surface is still unsafe. `IPermissionsAdapter.updateAllowedWrapper` only documents that a wrapper should expose `swap` or `modifyLiquidity`. It does not enforce that `msgSender()` is derived from a trusted lock or signed context. Consequently, approving a buggy or third-party wrapper can silently weaken the permission checks for every pool that trusts that adapter.

**Recommendation:** Treat `allowedWrappers` as a full trust list unless the hook can authenticate the reported user itself.

Cache `sender.msgSender()` once and reuse it for both currency checks so the two sides of the pool cannot be evaluated against different subjects. More importantly, only whitelist wrappers whose `msgSender()` is derived from protocol-controlled lock or signature state, or redesign the hook so the permission subject comes from trusted protocol state instead of from the wrapper contract.

**Uniswap:** Fixed in [PR 534](#) and [PR 551](#).

**Cantina Managed:** Fix verified.

### 3.4.3 Permissioned admins can seize LP NFTs

**Severity:** Informational

**Context:** [PermissionedPositionManager.sol#L59-L68](#)

**Description:** `PermissionedPositionManager.transferFrom(...)` gives the owner of either verified permissioned adapter in a pool unilateral transfer authority over the LP NFT. The function reads the admin for `currency0` and `currency1`, accepts either one as `msg.sender`, writes `getApproved[id] = msg.sender`, and then calls `super.transferFrom(...)`. The current NFT owner does not need to approve that transfer.

This is a full position seizure power, not only a freeze over the permissioned token. A v4 LP position is represented by one indivisible NFT. Therefore, in a `permissionedAdapter / USDC` pool, the permissioned-token admin can transfer Alice's LP NFT to itself. Once the admin owns the NFT, the inherited `_decrease(...)` and `_burn(...)` functions treat the admin as the approved position owner. The admin can then remove liquidity and receive both the permissioned-token side and the USDC side.

In a pool with two permissioned adapters, either adapter admin can unilaterally move the whole LP NFT. This may be intended for regulated-asset recovery or seizure. If so, it should be documented as a confiscation right because LPs are exposed to loss of the non-permissioned asset they paired with the regulated token. If adapter admins are not treated as fully trusted, the impact becomes direct privileged asset extraction from LPs.

**Recommendation:** Decide whether unilateral LP seizure by one permissioned-token admin is intended policy.

If it is intended, document the behavior explicitly for LPs and integrators. The disclosure should state that an adapter admin can take the entire LP NFT and later withdraw all assets represented by that position, including any non-permissioned token in the pool.



If it is not intended, require both NFT-owner approval and adapter-admin approval before moving a position. For two-permissioned-token pools, require both adapter admins. A safer recovery design is to move seized positions only to a dedicated escrow contract, split the non-permissioned proceeds back to the prior LP owner, or replace forced transfer with forced freeze.

**Uniswap:** Fixed in [PR 542](#) and [PR 543](#).

**Cantina Managed:** [PR 542](#) and [PR 543](#) fix this by removing the transferability of LP positions and enabling admins to unwind positions instead of seizing them.